MGL 7230: Tests Logiciels

Chapitre #2

Test de flux de contrôle

Plan

- Graphe de couverture
- Graphe de flux/flot de contrôle (Control Flow Graph)
- Couverture structurelle
  - Couverture des nœuds
  - Couverture des arêtes
  - Couverture des paires d'arêtes
  - Couverture du chemin spécifié (SPC)
  - Couverture du chemin premier
- Complexité cyclomatique
- Couverture des expressions logiques
  - Couverture de la clause active
  - Couverture générale de la clause active
  - Couverture corrélée de la clause active
  - Couverture restreinte de la clause active
- Exigences de test irréalisables

Graphes de couverture

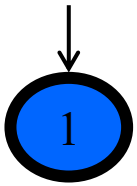
- Les graphiques sont la structure la plus couramment utilisée pour les tests
- Les graphiques peuvent provenir de nombreuses sources (graphiques de flux de contrôle, machines à états finis (FSM) et diagrammes d'état (UML state diagram, etc.)

Les tests sont généralement destinés à « couvrir » le graphique d'une manière ou d'une autre.

Définition d'un graphe

- Un ensemble N de nœuds, N n'est pas vide
- Un ensemble N_0 de nœuds initiaux, N_0 n'est pas vide
- Un ensemble N_f de nœuds finaux, N_f n'est pas vide
- Un ensemble E d'arêtes, chaque arête allant d'un nœud à un autre(n_i , n_j), i est le prédécesseur, j est le successeur

Est-ce un graphe ?

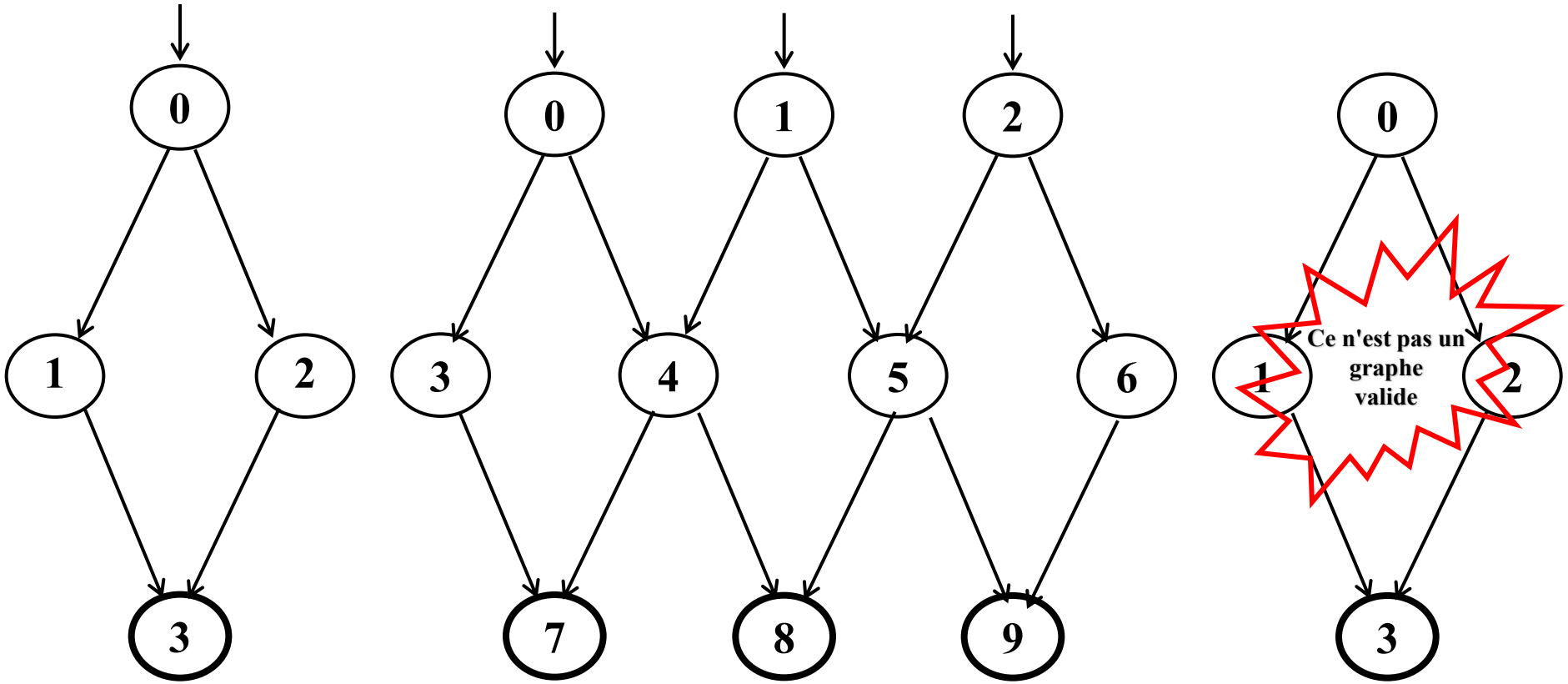


$N_0 = \{ 1 \}$

$N_f = \{ 1 \}$

$E = \{ \}$

Trois exemples de graphes



$N_0 = \{ 0 \}$

$N_f = \{ 3 \}$

$N_0 = \{ 0, 1, 2 \}$

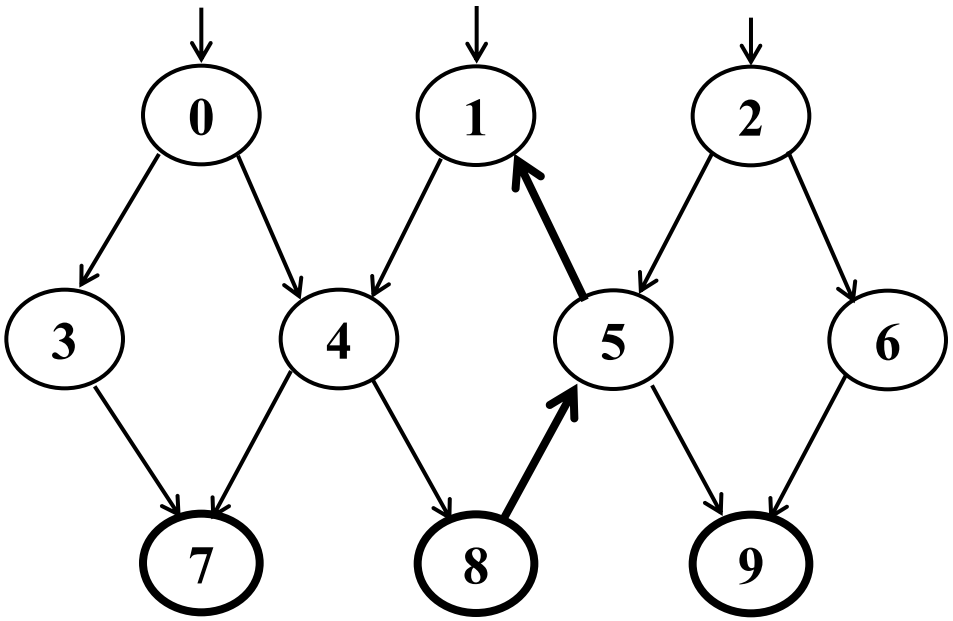
$N_f = \{ 7, 8, 9 \}$

$N_0 = \{ \}$

$N_f = \{ 3 \}$

Chemins dans les graphes

- Chemin : une séquence de nœuds – $[n_1, n_2, \ldots, n_M]$
  - Chaque paire de nœuds est une arête
- Longueur : Le nombre d'arêtes
  - Un seul nœud est un chemin de longueur 0
- Sous-chemin : Une sous-séquence de nœuds dans p est un sous-chemin de p
- Reach (n) (Portée (n)) : Sous-graphe accessible à partir de n



Exemples de chemins

[0, 3, 7]

[1, 4, 8, 5, 1]

[2, 6, 9]

Exemples d'accessibilité :

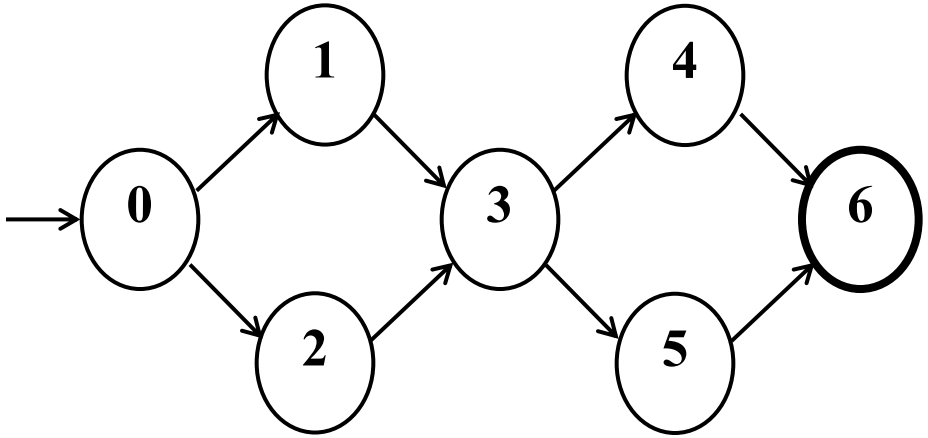
Reach (0) = { 0, 3, 4, 7, 8, 5, 1, 9 }

Reach ([0,1,2]) = N

Reach([6,9]) = { 6, 9 }

Chemins de test

- Chemin de test: un chemin p, éventuellement de longueur zéro, qui commence à un nœud **initial** et se termine à un nœud **final**
- Les chemins de test représentent l'exécution des cas de test
  - Certains chemins de test peuvent être exécutés par de nombreux tests
  - Certains chemins de test ne peuvent être exécutés par aucun test
- Graphe SESE : tous les chemins de test commencent à un nœud unique et se terminent à un nœud unique
  - Entrée unique, sortie unique
  - N_0 et N_f avoir exactement un nœud



Graphe à double losange

Quatre chemins de test

[0, 1, 3, 4, 6]
[0, 1, 3, 5, 6]
[0, 2, 3, 4, 6]
[0, 2, 3, 5, 6]

Visit and Touring

- Visit :

Un chemin de test p visite le nœud n si n fait partie de p

Un chemin de test p visite l'arête e si e fait partie de p

- Tour (Tournée) : Un chemin de test p parcourt le sous-chemin q si q est un sous-chemin de p

chemin [0, 1, 3, 4, 6]

Visites des nœuds 0, 1, 3, 4, 6

Visites des arêtes (0, 1), (1, 3), (3, 4), (4, 6)

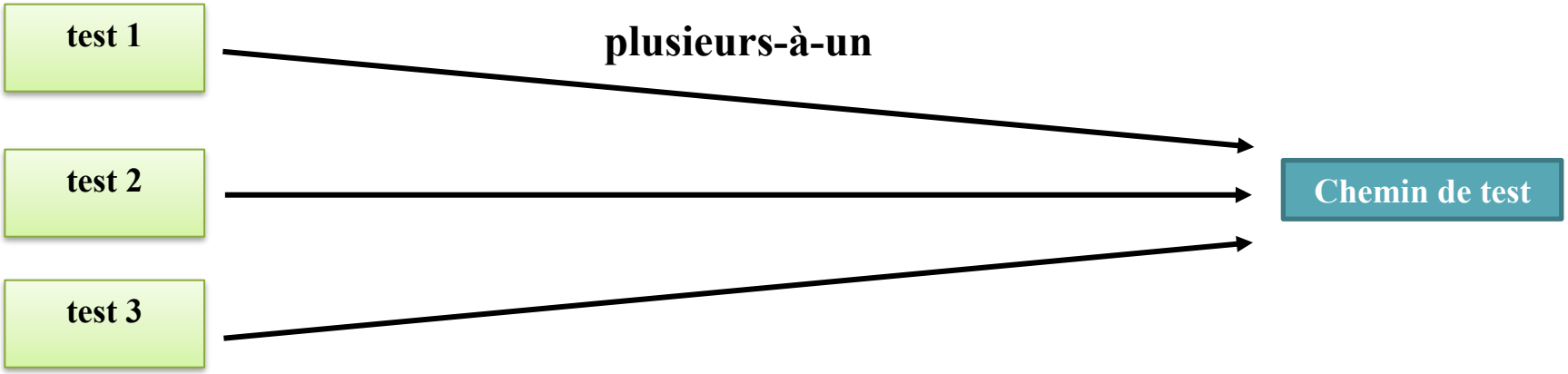
Tours sub-paths (0, 1, 3), (1, 3, 4), (3, 4, 6), (0, 1, 3, 4), (1, 3, 4, 6)

(De plus, chaque arête est techniquement un sous-chemin)

Tests et chemins de test

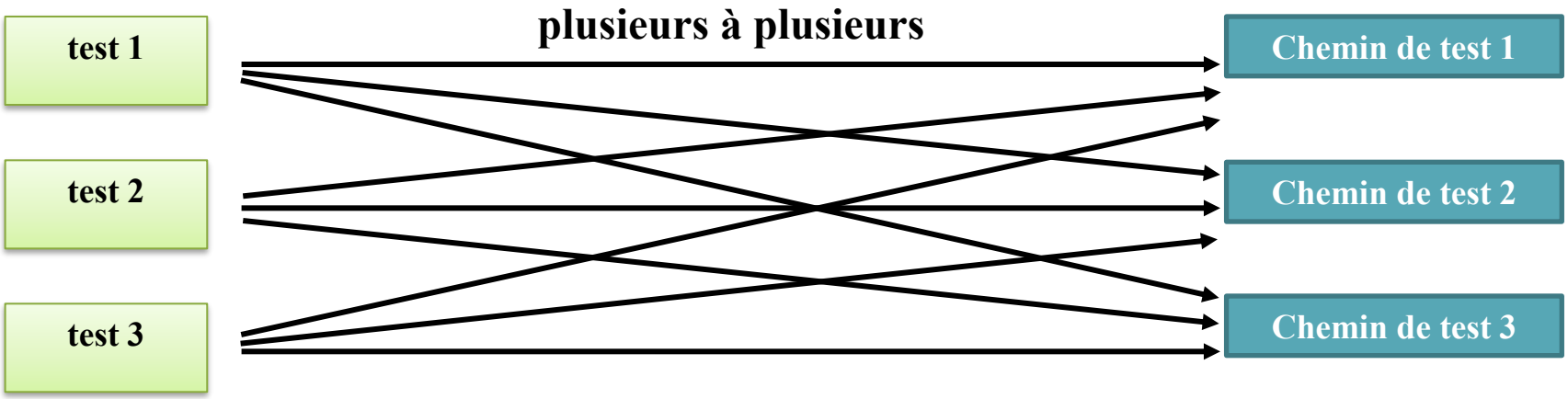
- path(t) : Le chemin de test exécuté par le test t
- path(T) : L'ensemble des chemins de test exécutés par l'ensemble des tests T
- Chaque test exécute un et un seul chemin de test
- Un emplacement dans un graphe (nœud ou arête) peut être atteint à partir d'un autre nœud s'il existe une séquence d'arêtes allant du premier emplacement au deuxième
  - *Portée syntaxique : un sous-chemin existe dans le graphe*
  - *Portée sémantique : un test existe qui peut exécuter ce sous-chemin*

Tests et chemins de test



Logiciel déterministe – un test exécute toujours le même chemin de test

Tests et chemins de test



Logiciel non déterministe – un test peut exécuter différents chemins de test

- Un algorithme non déterministe est un algorithme qui peut présenter des comportements différents selon les exécutions, contrairement à un algorithme déterministe.
- Un algorithme peut se comporter différemment d'une exécution à l'autre de plusieurs manières. Par exemple:
  - Un algorithme **concurrent** peut fonctionner différemment sur différentes exécutions en raison d'une condition de concurrence.
  - Les comportements d'un algorithme probabiliste dépendent d'un générateur de nombres aléatoires.

Partie I : Critères de couverture des graphiques

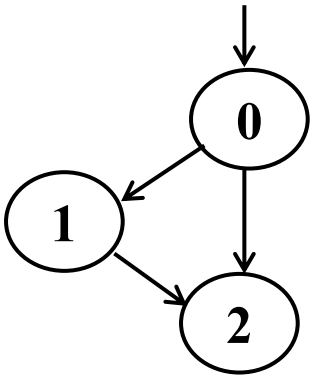
- Nous utilisons des graphes dans les tests comme suit :
  - Développer un modèle du logiciel sous forme de graphe
  - Exiger que les tests visitent ou parcourent des ensembles spécifiques de nœuds, d'arêtes ou de sous-chemins
- Critère de couverture : Règles qui définissent les exigences de test
- Exigences de test (Test Requirements TR) : Décrire les propriétés des chemins de test

1. Couverture des nœuds

- Le premier critère (et le plus simple) exige que chaque nœud d'un graphe soit exécuté

<u>Couverture des nœuds (Node Coverage (NC)) :</u>

TR contient chaque nœud accessible dans G.



<u>Couverture des nœuds</u> : TR = { 0, 1, 2 }
Chemin de test = [0, 1, 2]

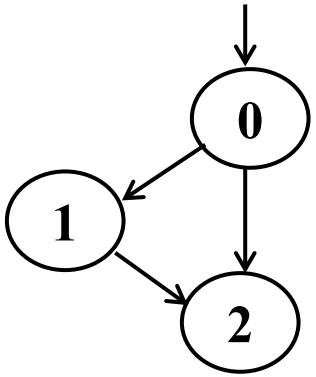
2. Couverture des arêtes

- La couverture des arêtes est légèrement plus forte que la couverture des nœuds

<u>Couverture des arêtes (Edge Coverage) :</u>

TR contient chaque chemin accessible d'une longueur allant jusqu'à 1, inclus, dans G.

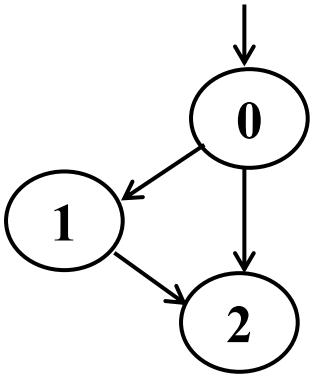
- La « longueur jusqu'à 1 » permet des graphes avec un seul nœud et aucune arête



<u>Couverture des arêtes</u> : TR = { (0,1), (0, 2), (1, 2) }
Test Paths = [0, 1, 2] [0, 2]

Couverture des nœuds et des périphériques

Node Coverage (NC) et Edge Coverage (EC) ne sont différents que lorsqu'il existe une arête et un autre sous-chemin entre une paire de nœuds. Notez que nous avons besoin de plus de chemins de test pour satisfaire le TR d'EC.



<u>Couverture des nœuds</u> : TR = { 0, 1, 2 }
Test Path = [0, 1, 2]

<u>Couverture des arêtes</u> : TR = { (0,1), (0, 2), (1, 2) }
Test Paths = [0, 1, 2] [0, 2]

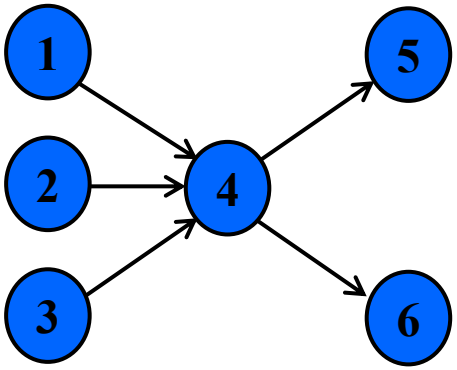
3. Couvrir plusieurs arêtes

- La couverture par paires d'arêtes nécessite des paires d'arêtes ou des sous-chemins de longueur 2

<u>Couverture des paires d'arêtes (Edge Pair Coverage) :</u>

TR contient chaque chemin accessible d'une longueur allant jusqu'à 2, inclus, dans G.

- La « longueur jusqu'à 2 » est utilisée pour inclure les graphiques qui ont moins de 2 arêtes



<u>Couverture des paires d'arêtes:</u>

<u>TR = { [1,4,5], [1,4,6], [2,4,5], [2,4,6], [3,4,5], [3,4,6] }</u>

4. Couverture du chemin

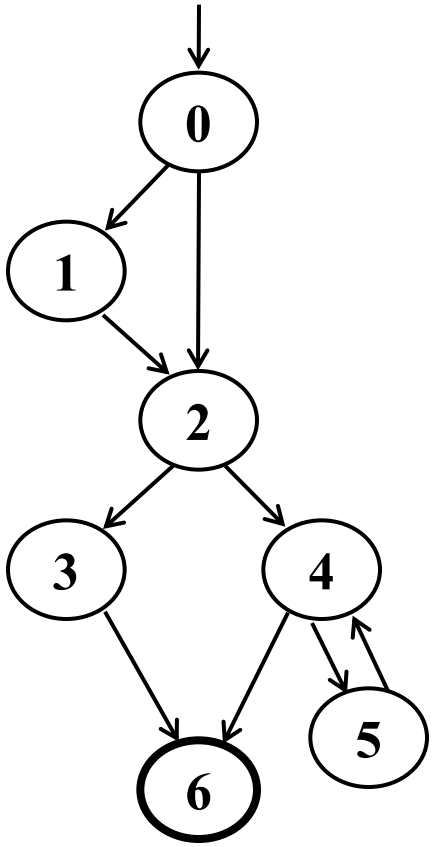
Couverture complète du chemin (Complete Path Coverage (CPC)) : TR contient tous les chemins dans G.

- Malheureusement, cela est impossible si le graphe comporte une boucle, donc **un compromis consiste à spécifier les chemins que nous souhaitons.**
- Ces chemins pourraient être fournis par le client sous forme de scénarios d'utilisation.

Couverture de chemin spécifiée (Specified Path Coverage (SPC)):

TR contient un ensemble S de chemins de test, où S est fourni comme paramètre.

Exemple de couverture structurelle



Couverture des nœuds (Node Coverage)

TR = { 0, 1, 2, 3, 4, 5, 6 }

Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 5, 4, 6]

Couverture des arêtes (Edge Coverage)

TR = { (0,1), (0,2), (1,2), (2,3), (2,4), (3,6), (4,5), (4,6), (5,4) }

Test Paths: [0, 1, 2, 3, 6] [0, 2, 4, 5, 4, 6]

Couverture des paires d'arêtes (Edge-pair coverage)

TR = { [0,1,2], [0,2,3], [0,2,4], [1,2,3], [1,2,4], [2,3,6], [2,4,5], [2,4,6], [4,5,4], [5,4,5], [5,4,6] }

Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 6] [0, 2, 3, 6] [0, 2, 4, 5, 4, 5, 4, 6]

Couverture complète du chemin (Complete Path Coverage)

Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 6] [0, 1, 2, 4, 5, 4, 6] [0, 1, 2, 4, 5, 4, 5, 4, 6] [0, 1, 2, 4, 5, 4, 5, 4, 5, 4, 6] …

Boucles dans les graphes

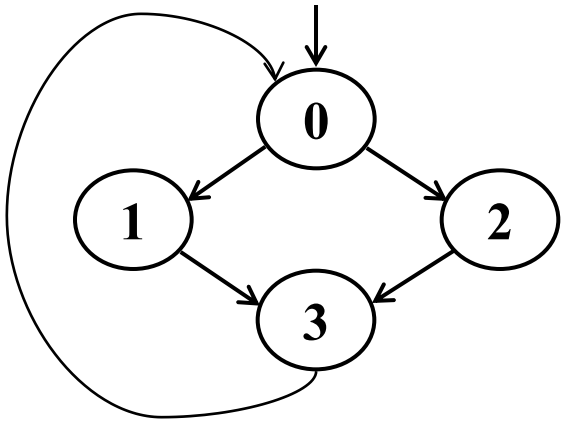
- Si un graphe contient une boucle, il possède un nombre infini de chemins
- Ainsi, la couverture complète du chemin (CPC) n'est pas réalisable.
- La couverture spécifiée du chemin (SPC) peut ne pas être satisfaisante car les résultats sont subjectifs et varient d'un système à l'autre.

Chemins simples (simple path) et chemins premiers (prime paths)

- Chemin simple : un chemin allant du nœud n_i au nœud n_j est simple si aucun nœud n'apparaît plus d'une fois, sauf si le premier et le dernier nœud sont identiques.
  - Aucune boucle interne.
  - Mais une boucle aller-retour est elle-même un chemin simple puisque le premier et le dernier nœud sont identiques.

Chemins simples et chemins premiers

- Chemin premier : un chemin simple qui n'apparaît pas comme un sous-chemin propre d'un autre chemin simple



Chemins simples (23 chemins) : [0, 1, 3, 0], [0, 2, 3, 0], [1, 3, 0, 1], [2, 3, 0, 2], [3, 0, 1, 3], [3, 0, 2, 3], [1, 3, 0, 2], [2, 3, 0, 1], [0, 1, 3], [0, 2, 3], [1, 3, 0], [2, 3, 0], [3, 0, 1], [3, 0, 2], [0, 1], [0, 2], [1, 3], [2, 3], [3, 0], [0], [1], [2], [3]

Chemins premiers (8 chemins) : [0, 1, 3, 0], [0, 2, 3, 0], [1, 3, 0, 1], [2, 3, 0, 2], [3, 0, 1, 3], [3, 0, 2, 3], [1, 3, 0, 2], [2, 3, 0, 1]

Couverture du chemin premier

- Un critère simple, élégant et fini qui nécessite que les boucles soient exécutées mais pas à l'infini.

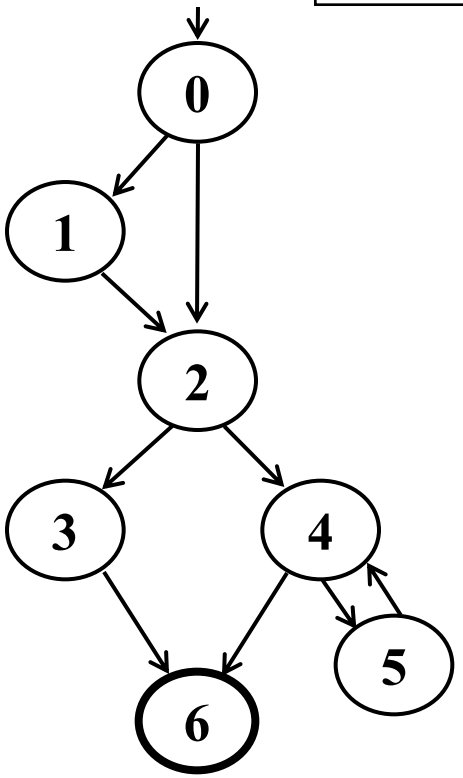
Couverture du chemin premier (Prime Path Coverage (PPC)) :

TR contient chaque chemin premier dans G.

Trouver les chemins premiers

« ! » signifie que le chemin se termine

« * » signifie cycles de chemin



Simple paths

Len 0
[0]
[1]
[2]
[3]
[4]
[5]
[6] !

Len 1
[0, 1]
[0, 2]
[1, 2]
[2, 3]
[2, 4]
[3, 6] !
[4, 6] !
[4, 5]
[5, 4]

Len 2
[0, 1, 2]
[0, 2, 3]
[0, 2, 4]
[1, 2, 3]
[1, 2, 4]
[2, 3, 6] !
[2, 4, 6] !
[2, 4, 5] *
[4, 5, 4] *
[5, 4, 6] !
[5, 4, 5] *

Len 3
[0, 1, 2, 3]
[0, 1, 2, 4]
[0, 2, 3, 6] !
[0, 2, 4, 6] !
[0, 2, 4, 5] *
[1, 2, 3, 6] !
[1, 2, 4, 5] *
[1, 2, 4, 6] !

Len 4
[0, 1, 2, 3, 6] !
[0, 1, 2, 4, 6] !
[0, 1, 2, 4, 5] *

Chemins premiers

Exigences de test irréalisables

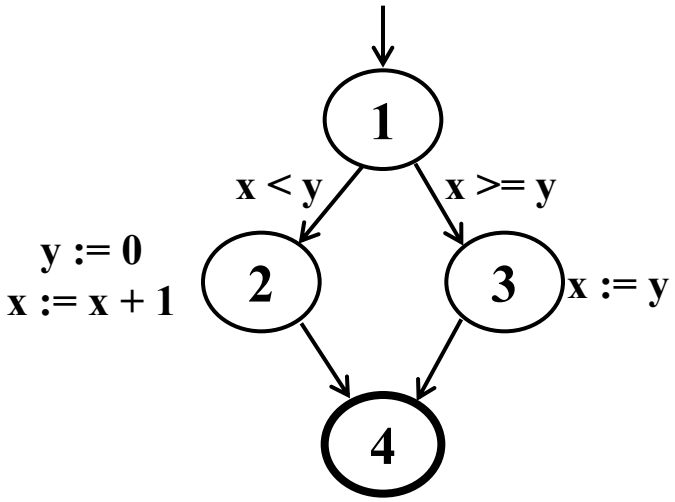
- Une exigence de test irréalisable ne peut pas être satisfaite, par exemple :Unreachable statement (dead code).
  - Un sous-chemin qui ne peut être exécuté que si une contradiction se produit (x<0 && x>0).
- La plupart des critères de test comportent des exigences de test irréalisables.
- Il est généralement impossible de déterminer si toutes les exigences de test sont réalisables.

Partie II : Graphiques de flux de contrôle (CFG) – Code source

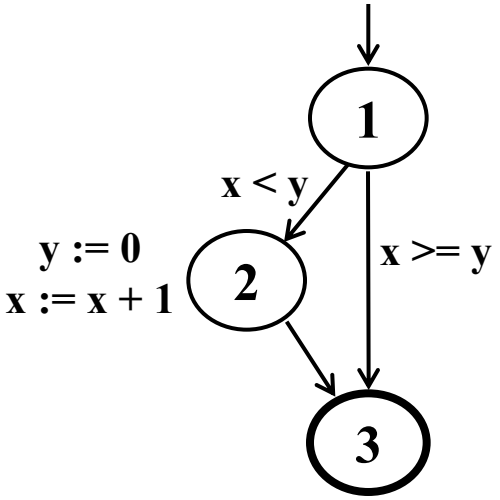
- Un CFG modélise toutes les exécutions d'une méthode en décrivant les structures de contrôle
  - Nœuds : instructions ou séquences d'instructions (blocs de base)
  - Arêtes : transferts de contrôle
- Un bloc de base dans le programme P est une séquence d'instructions consécutives avec une seule entrée et un seul point de sortie (pas de branches).
- Le contrôle entre toujours dans un bloc de base à son point d'entrée et sort de son point de sortie. Il n'y a aucune possibilité de sortie ou d'arrêt à n'importe quel point à l'intérieur du bloc de base, sauf à son point de sortie.
- Les CFG sont parfois annotés avec des informations supplémentaires
  - prédicats de branche
  - defs (définitions de variables), Uses (utilisations de variables)

CFG : L'instruction if

```
if (x < y)
{
   y := 0;
   x := x + 1;
}
else
{
   x := y;
}
```

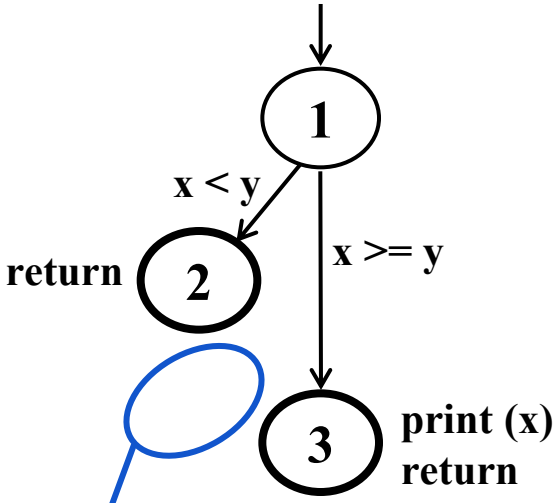



```
if (x < y)
{
   y := 0;
   x := x + 1;
}
```



CFG : L'instruction if-Return

```
if (x < y)
{
   return;
}
print (x);
return;
```



Pas d'arête entre les nœuds 2 et 3.
Les nœuds de retour doivent être distincts.

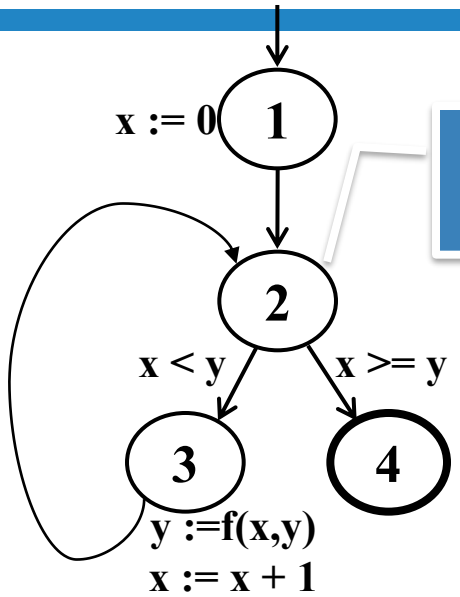
Boucles

- Les boucles nécessitent l'ajout de nœuds « supplémentaires »

- Nœuds qui ne représentent pas d'instructions ou de blocs de base

CFG : boucles while et for

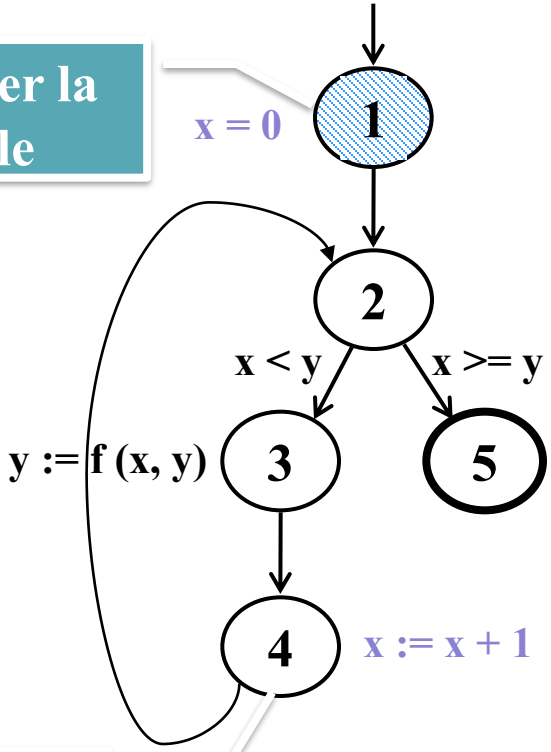
```
x := 0;
while (x < y)
{
   y := f (x, y);
   x := x + 1;
}
```



dummy node

Initialiser la boucle

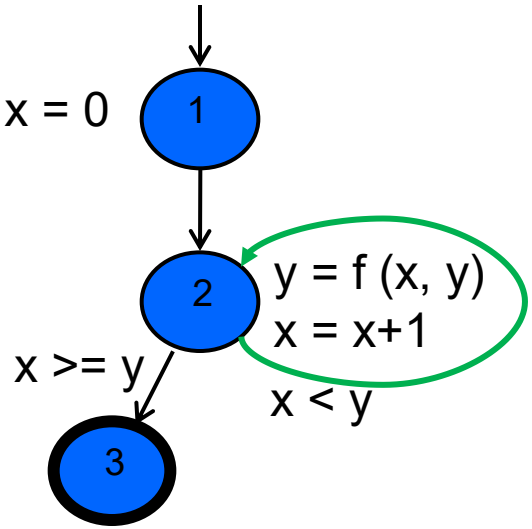
```
for (x = 0; x < y; x++)
{
   y := f (x, y);
}
```



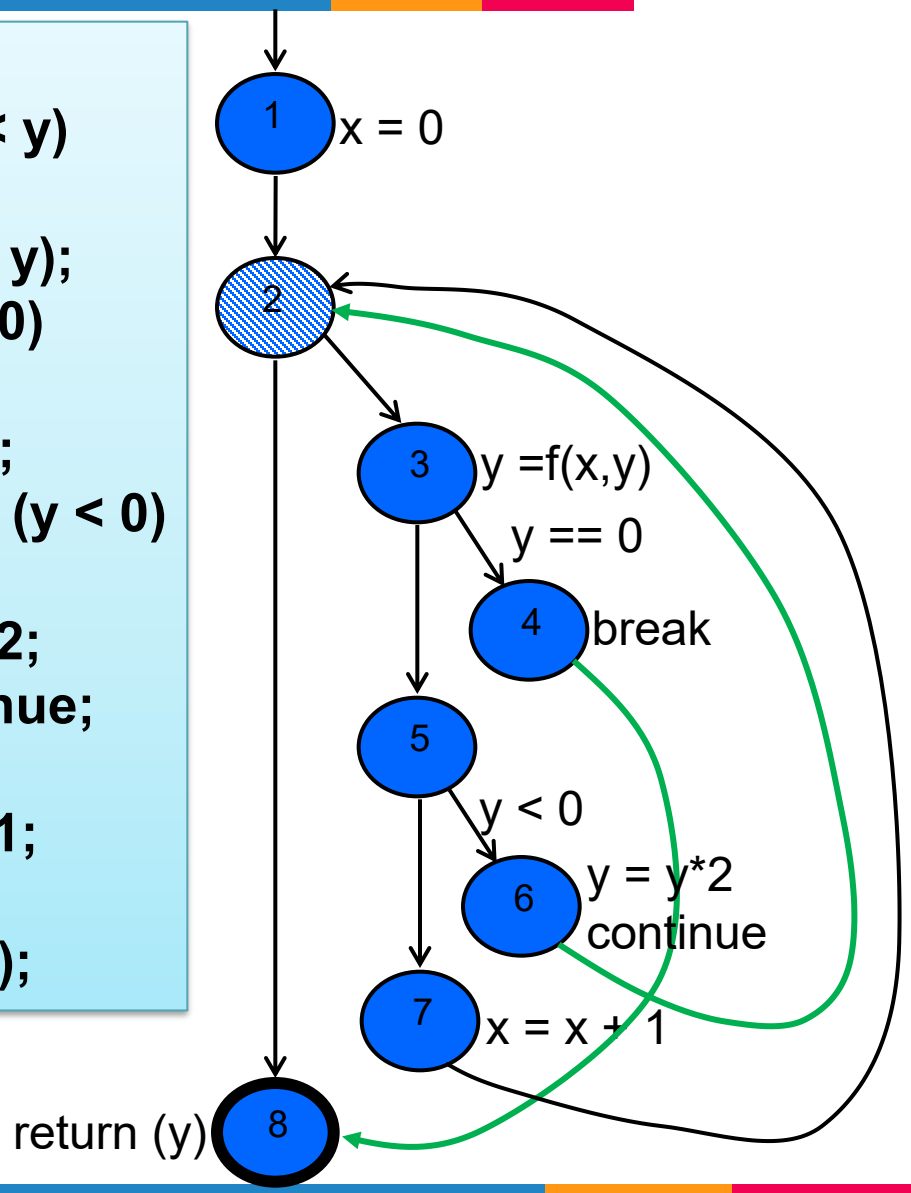
incrémente la boucle

CFG : do Loop, break and continue

```
x = 0;
do
{
   y = f (x, y);
   x = x + 1;
} while (x < y);
return  (y);
```

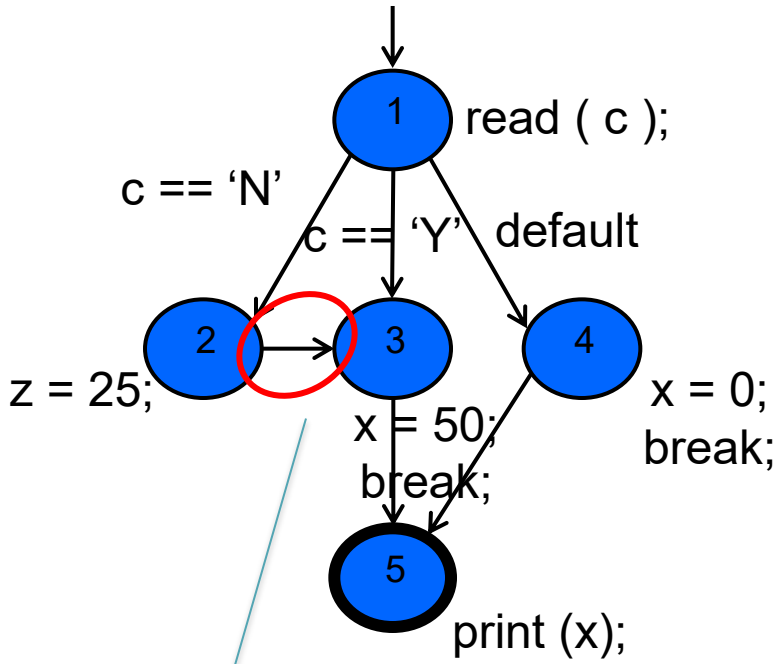



```
x = 0;
while (x < y)
{
   y = f (x, y);
   if (y == 0)
   {
      break;
   } else if (y < 0)
   {
      y = y*2;
      continue;
   }
   x = x + 1;
}
return  (y);
```



CFG : The case (switch) Structure

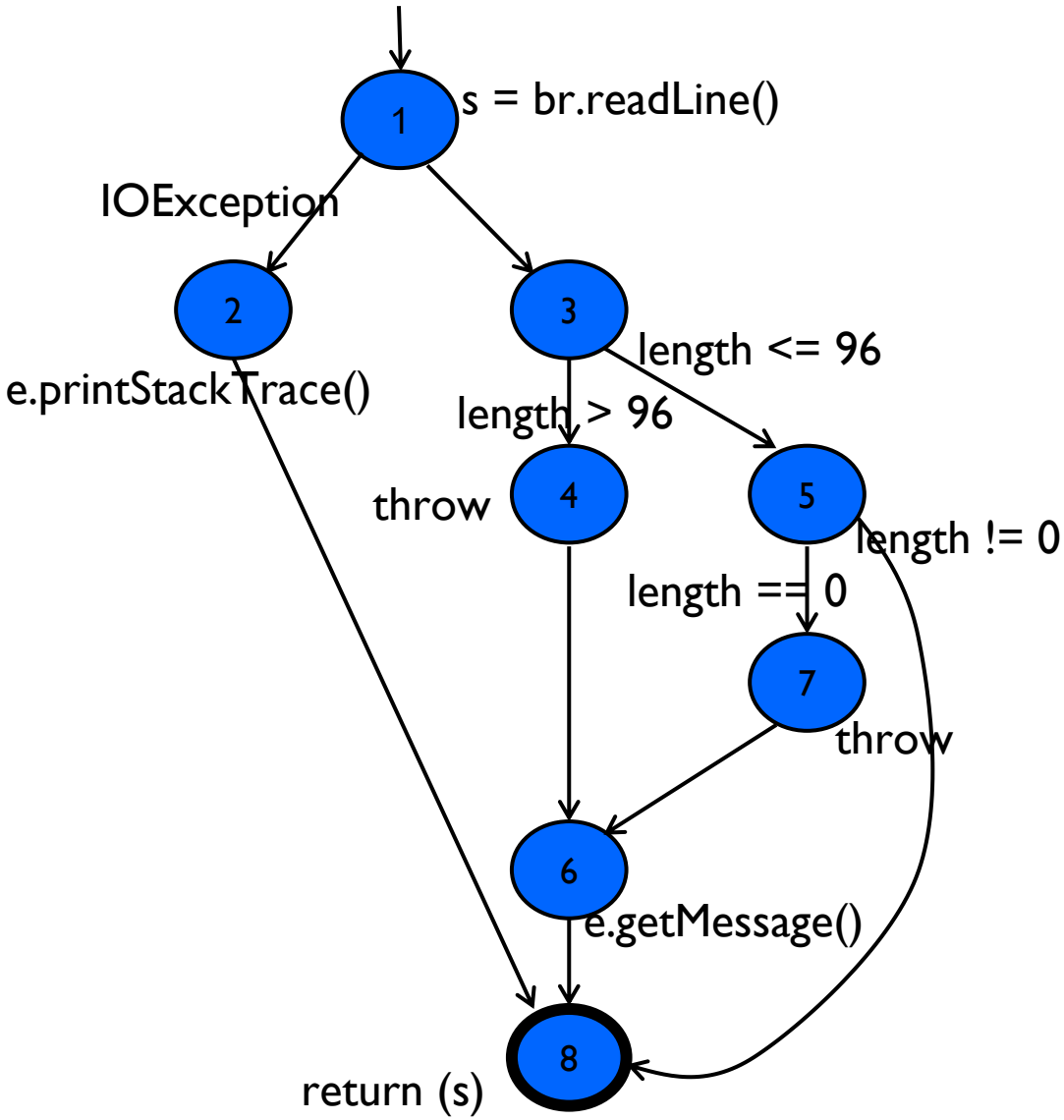
```
read ( c) ;
switch ( c )
{
   case 'N':
      z = 25;
   case 'Y':
      x = 50;
      break;
   default:
      x = 0;
      break;
}
print (x);
```



Cases without breaks fall through to the next case

CFG : Exceptions (try-catch)

```
try
{
   s = br.readLine();
   if (s.length() > 96)
      throw new Exception
         ("too long");
   if (s.length() == 0)
      throw new Exception
         ("too short");
} (catch IOException e) {
   e.printStackTrace();
} (catch Exception e) {
   e.getMessage();
}
return (s);
```

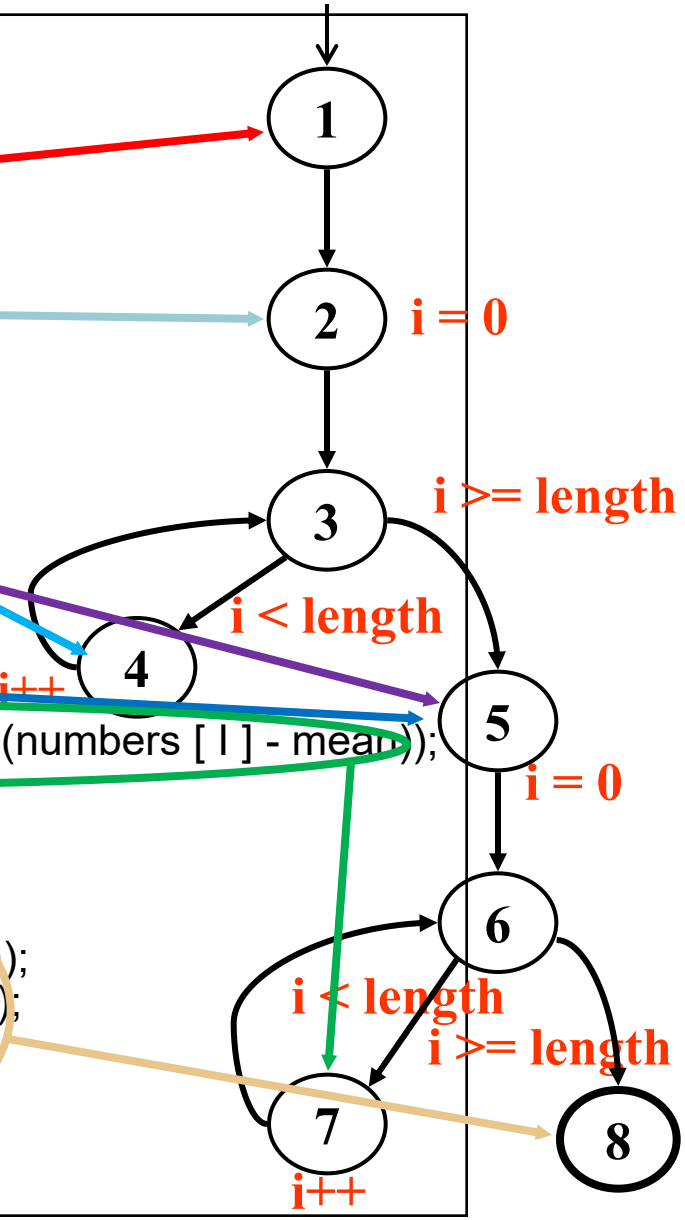


Exemple de flux de contrôle – Statistiques

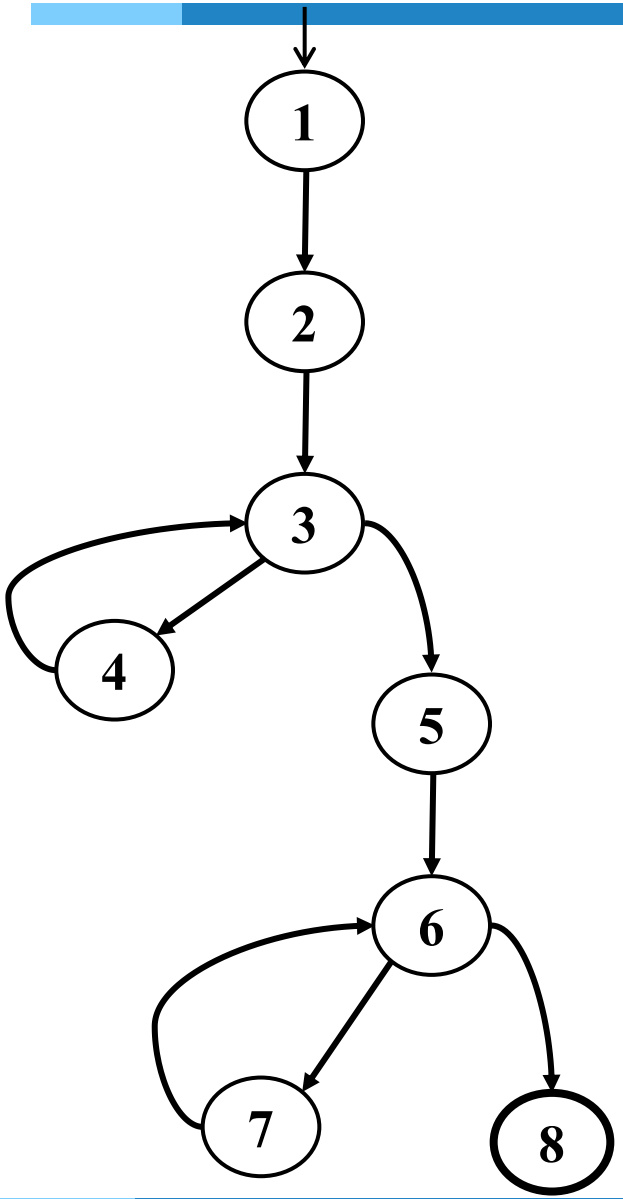
```
public static void computeStats (int [ ] numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;

    sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med   = numbers [ length / 2 ];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum  + ((numbers [ I ] - mean) * (numbers [ I ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length:                    " + length);
    System.out.println ("mean:                      " + mean);
    System.out.println ("median:                    " + med);
    System.out.println ("variance:                   " + var);
    System.out.println ("standard deviation: " + sd);
}
```

Graphique de flux de contrôle pour les statistiques

```
public static void computeStats (int [ ] numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;

    sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med   = numbers [ length / 2 ];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum  + ((numbers [ I ] - mean) * (numbers [ I ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length:                    " + length);
    System.out.println ("mean:                      " + mean);
    System.out.println ("median:                    " + med);
    System.out.println ("variance:                  " + var);
    System.out.println ("standard deviation: " + sd);
}
```



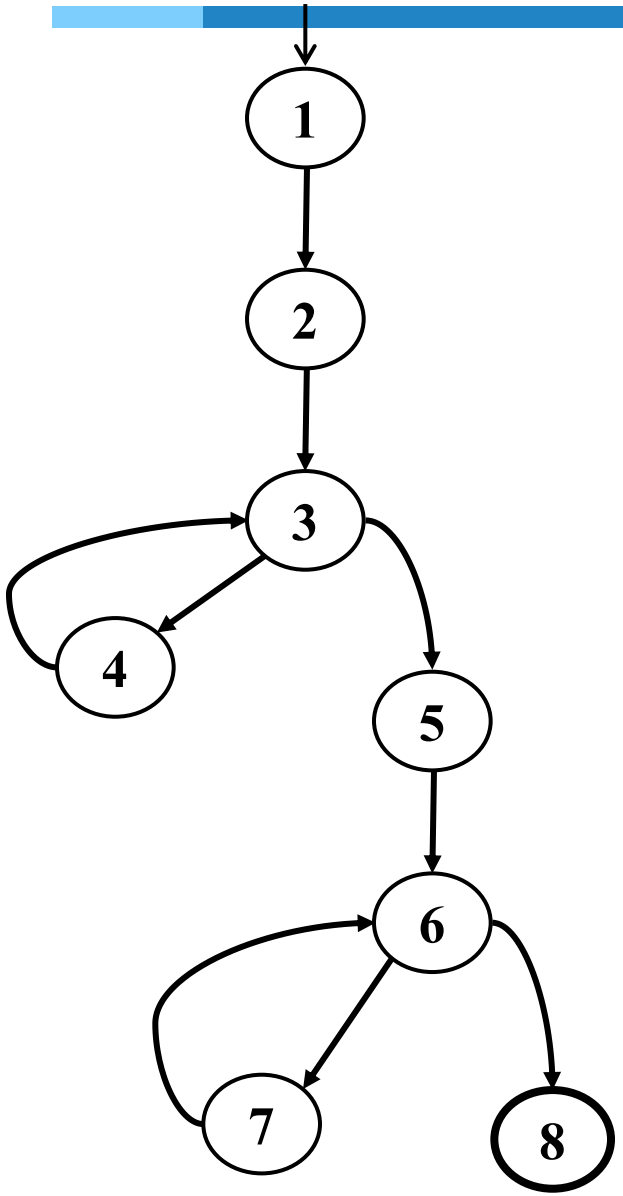
Contrôle des flux TR et chemins de test – EC



Couverture des arêtes

TR	Chemin de test
A. [1, 2] B. [2, 3] C. [3, 4] D. [3, 5] E. [4, 3] F. [5, 6] G. [6, 7] H. [6, 8] I. [7, 6]	[1, 2, 3, 4, 3, 5, 6, 7, 6, 8]

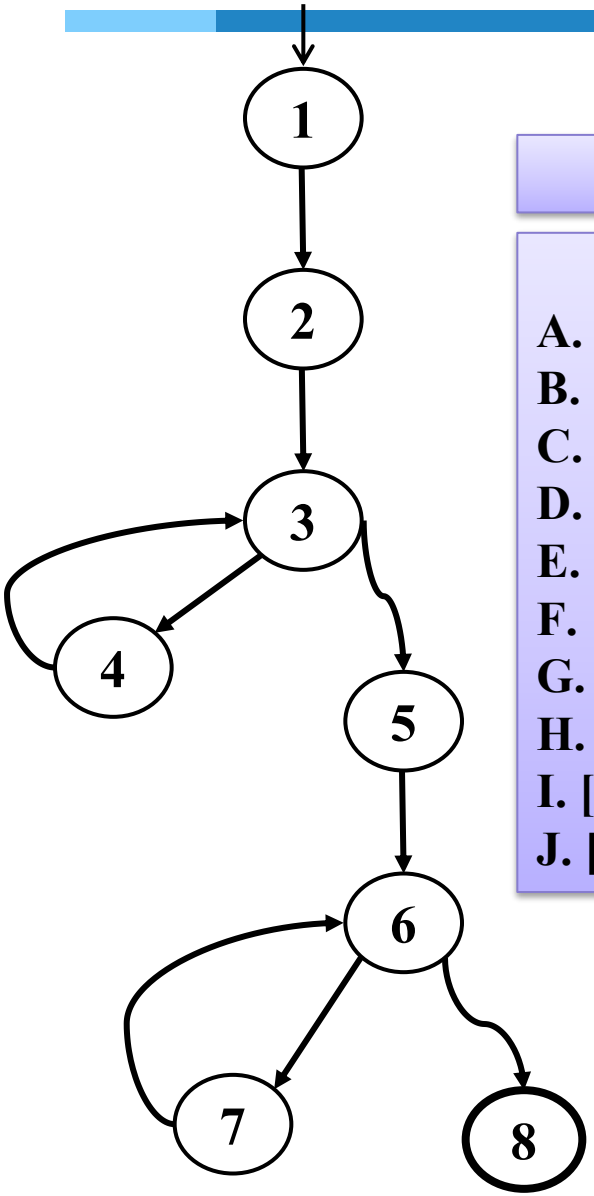
Contrôle des flux TR et chemins de test – EPC



Couverture des paires d'arêtes

TR	Chemins de test
A. [1, 2, 3]	i. [1, 2, 3, 4, 3, 5, 6, 7, 6, 8]
B. [2, 3, 4]	ii. [1, 2, 3, 5, 6, 8]
C. [2, 3, 5]	iii. [1, 2, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 8]
D. [3, 4, 3]	
E. [3, 5, 6]	
F. [4, 3, 5]	
G. [5, 6, 7]	
H. [5, 6, 8]	
I. [6, 7, 6]	
J. [7, 6, 8]	
K. [4, 3, 4]	
L. [7, 6, 7]	

Contrôler les flux TR et les chemins de test – PPC



Couverture du chemin premier

TR

A. [3, 4, 3]
B. [4, 3, 4]
C. [7, 6, 7]
D. [7, 6, 8]
E. [6, 7, 6]
F. [1, 2, 3, 4]
G. [4, 3, 5, 6, 7]
H. [4, 3, 5, 6, 8]
I. [1, 2, 3, 5, 6, 7]
J. [1, 2, 3, 5, 6, 8]

Chemins de test

i. [1, 2, 3, 4, 3, 5, 6, 7, 6, 8]
ii. [1, 2, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 8]
iii. [1, 2, 3, 4, 3, 5, 6, 8]
iv. [1, 2, 3, 5, 6, 7, 6, 8]
v. [1, 2, 3, 5, 6, 8]

Exercice

Le programme P suivant prend en entrée deux entiers x et y et génère en sortie xy.

1. Identifier les blocs de base de P
2. Construire le graphe de contrôle de P

```
1 begin
2   int x, y, power;
3   float z;
4   input (x, y);
5   if (y<0)
6     power=-y;
7   else
8     power=y;
9   z=1;
10  while (power!=0){
11    z=z*x;
12  power=power-1;
13  }
14  if (y<0)
15    z=1/z;
16  output(z);
17 end
```

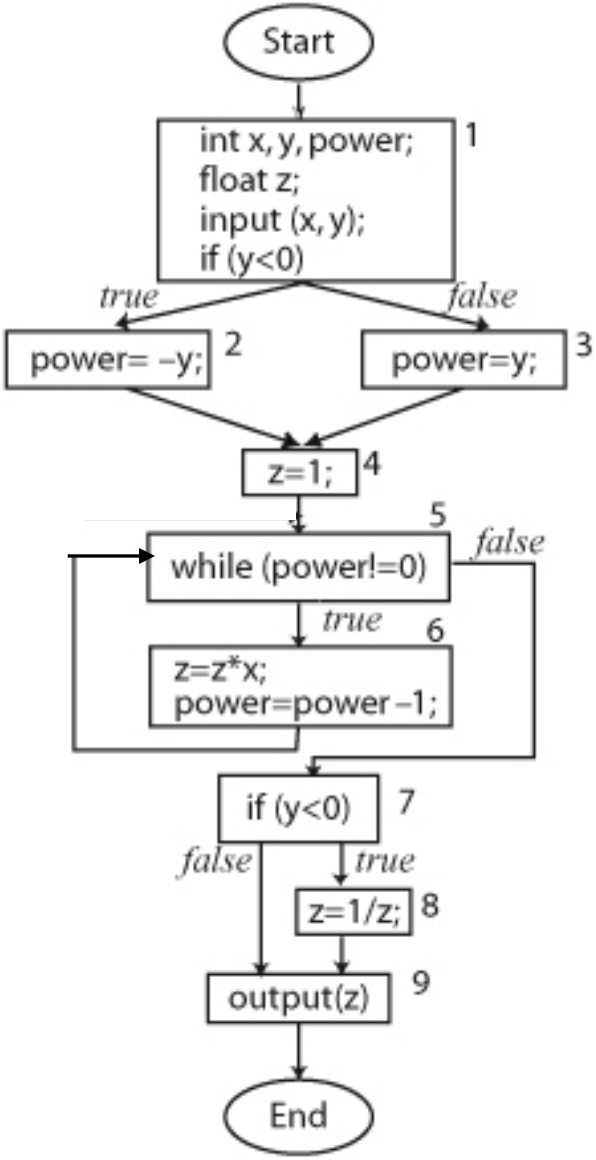
Solution de l'exercice

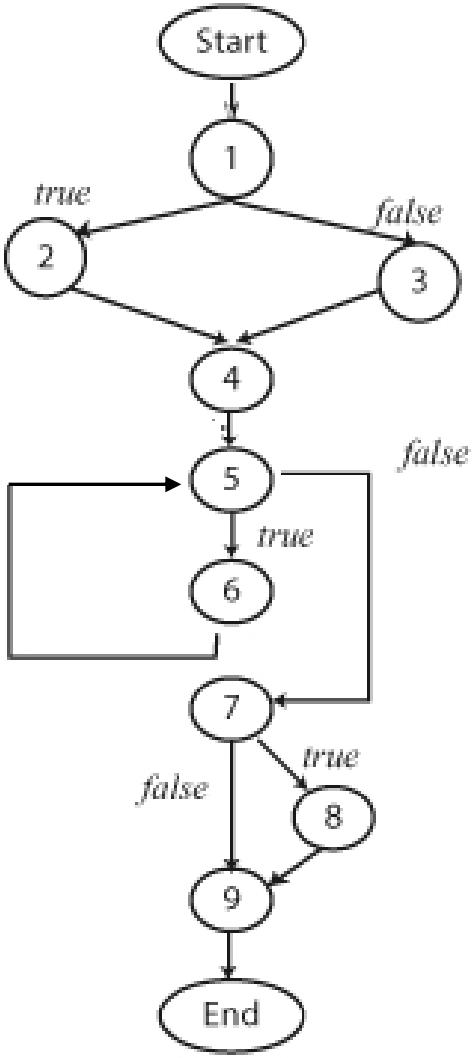
```
 1 begin
 2   int x, y, power;
 3   float z;
 4   input (x, y);
 5   if (y<0)
 6     power=-y;
 7   else
 8     power=y;
 9   z=1;
10   while (power!=0){
11     z=z*x;
12   power=power-1;
13   }
14   if (y<0)
15     z=1/z;
16   output(z);
17 end
```

Définitions des blocs

Block	Lines	Entry Point	Exit Point
1	2,3,4,5	1	5
2	6	6	6
3	8	8	8
4	9	9	9
5	10	10	10
6	11, 12	11	12
7	14	14	14
8	15	15	15
9	16	16	16

```
1  begin
2    int x, y, power;
3    float z;
4    input (x, y);
5    if (y<0)
6      power=-y;
7    else
8      power=y;
9    z=1;
10   while (power!=0){
11     z=z*x;
12   power=power-1;
13   }
14   if (y<0)
15     z=1/z;
16   output(z);
17 end
```





N = {Start, 1, 2, 3, 4, 5, 6, 7, 8, 9, End}

E = {(Start, 1), (1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (5, 6), (6, 5), (5, 7), (7, 8), (7, 9), (9, End)}

Mesurer la complexité des logiciels

- Mesure de la complexité informatique grande O, O(n)
  - Mesure la complexité du logiciel du point de vue de la machine en termes de la façon dont la taille des données d'entrée affecte l'utilisation des ressources informatiques d'un algorithme (généralement le temps d'exécution ou la mémoire)

- La mesure de la complexité en ingénierie logicielle doit mesurer la complexité du point de vue des développeurs humains
  - Le temps informatique est bon marché, le temps humain est cher

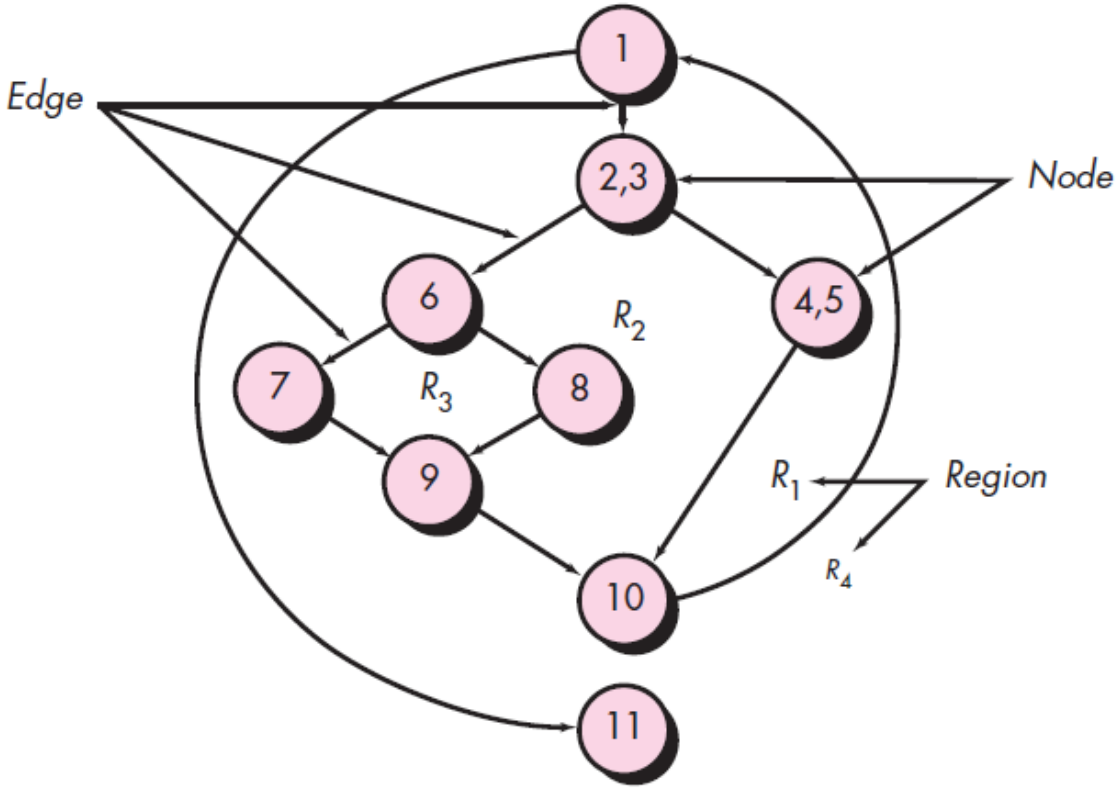
Complexité cyclomatique

- Inventé par Thomas McCabe (1974) pour mesurer la complexité de la logique conditionnelle d'un programme
  - Compte le nombre de décisions dans le programme, en supposant que les décisions sont difficiles pour les gens
- Calculée de trois façons:

1. V(G) = e – n + 2; e est #arêtes et n est #nœuds
2. Le nombre de régions du CFG
3. V(G) = P +1; où P est le nombre de nœuds de prédicat

(Notez que vous n'avez pas besoin de construire le CFG pour utiliser cette méthode)

Complexité cyclomatique V(G)



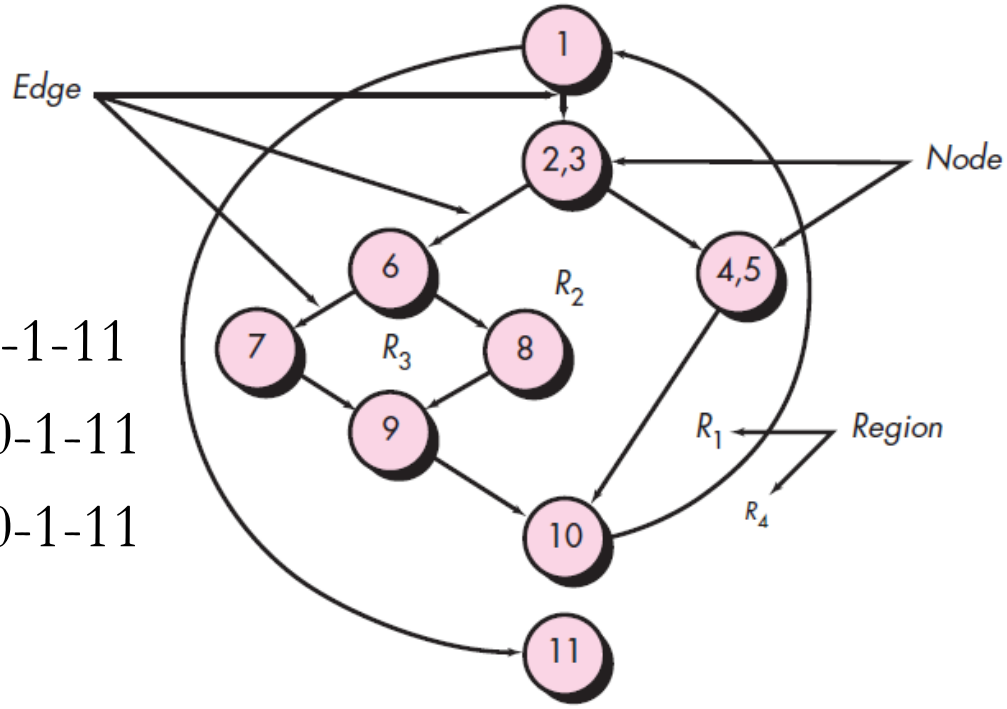
1. V(G) = 11 arêtes - 9 nœuds +2 = 4.
2. Le CFG compte quatre régions.
3. V(G)= 3 nœuds de prédicat + 1 = 4.

Test de boîte blanche : couverture du chemin

- V(G) fournit le nombre de chemins linéairement indépendants pour couvrir toutes **les instructions d'une méthode.**
- un **chemin linéairement indépendant** (*linearly independent path*) est un **chemin d'exécution de bout en bout** (de l'entrée à la sortie du graphe de flot de contrôle) qui **introduit au moins une nouvelle arête** (ou un nouveau segment) **qui n'apparaît dans aucun autre chemin indépendant déjà sélectionné**.
- Autrement dit : chaque chemin indépendant ajoute "quelque chose de nouveau" au **jeu de base** (*basis set*) de chemins.
- Concevez des tests pour garantir que chaque instruction du programme **sera exécutée au moins une fois et que chaque condition sera exécutée sur ses côtés vrai et faux.**
- V(G) >=1; G aura un chemin, si V(G) = 1

Chemins de test linéairement indépendants

- **Chemin 1 :** 1-11
- **Chemin 2:** 1-(2-3)-(4-5)-10-1-11
- **Chemin 3:** 1-(2-3)-6-8-9-10-1-11
- **Chemin 4:** 1-(2-3)-6-7-9-10-1-11



Notez que le chemin : 1-(2-3)-(4-5)-10-1-(2-3)-6-8-9-10-1-11 n'est pas un chemin indépendant car il ne traverse aucune nouvelle arête.

Interprétation des valeurs de complexité cyclomatique (CC)

- La valeur de 10 est souvent considérée comme le seuil entre un code acceptable (à faible risque) et un code trop complexe (à risque plus élevé).
- Recommandations (de l'outil d'analyse statique ndepend) :
  - Les méthodes où le CC est supérieur à 15 sont difficiles à comprendre et à maintenir.
  - Les méthodes où le CC est supérieur à 30 sont extrêmement complexes et doivent être divisées en méthodes plus petites (sauf si elles sont générées automatiquement par un outil).

https://www.ndepend.com/docs/code-metrics#CC

Partie III : Couvrir les expressions logiques

- La couverture des expressions logiques est exigée par la Federal Aviation Administration des États-Unis pour les logiciels critiques pour la sécurité.
- Les expressions logiques peuvent provenir de nombreuses sources :
  - **Décisions dans les programmes :**

```
If ((a > b) || C) && (x < y))
    o.m();
else
    o.n();
```

  - Gardes dans les machines à états finis et les diagrammes d'états
  - Décisions dans les graphes d'activité UML
  - Exigences, formelles et informelles
  - Requêtes SQL
- Les tests sont destinés à choisir **un sous-ensemble du nombre** total **d'affectations de vérité aux expressions**

Prédicats et clauses logiques

- Un prédicat est une expression qui s'évalue à une valeur booléenne
- Les prédicats peuvent contenir
  - variables booléennes
  - variables non booléennes contenant >, <, ==, >=, <=, !=
  - appels de fonctions booléennes
- La structure interne est créée par des opérateurs logiques
  - $\neg$ – l'opérateur de négation
  - $\wedge$ – l'opérateur et
  - $\vee$ – l'opérateur ou
  - $\rightarrow$ – l'opérateur d'implication
  - $\oplus$ – l'exclusif ou l'opérateur
  - $\leftrightarrow$ – l'opérateur d'equivalence
- Une clause est un prédicat sans opérateurs logiques

Exemples

$P : (a < b) \vee f(z) \wedge D \wedge (m >= n*o)$

- Quatre clauses :
  - $(a < b)$ – expression relationnelle
  - $f(z)$ – fonction à valeur booléenne
  - D – variable booléenne
  - $(m >= n*o)$ – expression relationnelle
- La plupart des prédicats ont peu de clauses
  - Il serait intéressant de quantifier cette affirmation !

Tester et couvrir les prédicats

- Nous utilisons des prédicats dans les tests comme suit :
  - Développer un modèle du logiciel sous la forme d'un ou plusieurs prédicats
  - Exiger des tests pour satisfaire une combinaison de clauses
- Abréviations :
  - P est l'ensemble des prédicats
  - p est un prédicat unique dans P
  - C est l'ensemble des clauses dans P
  - C_p est l'ensemble des clauses du prédicat p
  - c est une clause unique dans C

Couverture des prédicats et des clauses

- Les deux premiers critères (et les plus simples) exigent que chaque prédicat et chaque clause soient évalués à la fois comme vrai et faux.

Couverture des prédicats (PC) : Pour chaque p dans P, TR contient deux exigences : p est évalué à vrai et p est évalué à faux.

- Lorsque les prédicats proviennent de conditions sur les arêtes, cela équivaut à une couverture des arêtes
- PC n'évalue pas toutes les clauses, donc…

Couverture de clause (CC) : Pour chaque c dans C, TR contient deux exigences : c est évalué à vrai et c est évalué à faux.

Exemple de couverture de prédicat

$((a < b) \vee D) \wedge (m >= n*o)$

couverture de prédicat

prédicat = true

a = 5, b = 10, D = vraie, m = 1, n = 1, o = 1

$= (5 < 10) \vee$ vraie $\wedge (1 >= 1*1)$

= vraie $\vee$ vraie $\wedge$ VRAIE

= vraie

prédicat = FAUX

a = 10, b = 5, D = FAUX, m = 1, n = 1, o = 1

$= (10 < 5) \vee$ FAUX $\wedge (1 >= 1*1)$

= FAUX $\vee$ FAUX $\wedge$ VRAIE

= FAUX

Exemple de couverture de clauses

$((a < b) \vee D) \wedge (m >= n*o)$

Couverture des clauses

(a < b) = vraie	(a < b) = FAUX
a = 5, b = 10	a = 10, b = 5

D = vraie	D = FAUX
D = vraie	D = FAUX

m >= n*o = vraie	m >= n*o = FAUX
m = 1, n = 1, o = 1	m = 1, n = 2, o = 2

FAUX cases

vraie cases

Deux tests

1) a = 5, b = 10, D = vraie, m = 1, n = 1, o = 1

2) a = 10, b = 5, D = FAUX, m = 1, n = 2, o = 2

Problèmes avec PC et CC

- PC n'exerce pas pleinement toutes les clauses
- CC ne garantit pas toujours PC
  - Autrement dit, nous pouvons satisfaire CC sans que le prédicat soit à la fois vrai et faux.
  - a<b = vrai, D= vrai, m>=n*o = faux ➔ P= faux
  - a<b = faux, D= faux, m>=n*o = vrai ➔ P= faux

- Ce n'est certainement pas ce que nous souhaitons !
- La solution la plus simple est de tester toutes les combinaisons

…

Couverture combinatoire

La CoC exige toutes les combinaisons possibles

Parfois appelée couverture de conditions multiples

Couverture combinatoire (CoC) : Pour chaque p dans P, TR a des exigences de test pour les clauses de Cp à évaluer pour chaque combinaison possible de valeurs de vérité.

	a < b	D	m >= n*o	((a < b) ∨ D) ∧ (m >= n*o)
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	F
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Couverture combinatoire

C'est simple, net, propre et complet…

- Mais assez cher !
- 2^N tests, où N est le nombre de clauses
  - Impraticable pour les prédicats avec plus de 3 ou 4 clauses
- La littérature regorge de suggestions, certaines sont déroutantes
- L'idée générale est simple :

Testez chaque clause indépendamment des autres clauses

- Que signifie exactement « indépendamment » ?

Clauses actives

- La clause de couverture a une faiblesse : les valeurs ne font pas toujours la différence
- Considérez le premier test de couverture de clause, qui a conduit à ce que chaque clause soit vraie :
  - $((a < b) \vee true) \wedge (m >= n*o)$
- Seule la dernière clause compte !
- Pour vraiment tester les résultats d'une clause, la clause doit être le facteur déterminant de la valeur du prédicat

Determination : **Une clause c_i dans le prédicat p, appelée la clause majeure, détermine p si et seulement si les valeurs des clauses mineures restantes c_j sont tels que changer c_i modifie la valeur de p**

Ceci est considéré comme rendant la clause active

Déterminer les prédicats

- Objectif : **Trouver des tests pour chaque clause lorsque la clause détermine la valeur du prédicat.**
- Pour satisfaire à un critère donné, chaque clause est traitée à tour de rôle comme une clause majeure.
- Cela se formalise en **plusieurs critères** qui présentent des différences subtiles, mais très importantes.

$P = A \vee B$

Si *B = vraie*, *p* est toujours vrai.

donc Si *B = faux*, *A détermine p.*

Si *A = faux*, *B* A détermine p.

$P = A \wedge B$

Si *B = faux*, *p* est toujours faux.

Donc si *B = vraie*, *A détermine p*.

Si *A = vraie*, *B détermine p*.

Faire en sorte que les clauses déterminent un prédicat

- Comment trouver des valeurs pour les clauses mineures c_j ?
- Approche :
  - *pc=true est le prédicat p avec chaque occurrence de c remplacée par true*
  - *pc=false est le prédicat p avec chaque occurrence de c remplacée par false*
- Pour trouver les valeurs des clauses mineures, connectez $p_{c=true}$ et $p_{c=false}$ avec OU exclusif

$$p_c = p_{c=true} \oplus p_{c=false}$$

- Après avoir résolu, p_c décrit exactement les valeurs nécessaires pour que c détermine p

Exemples

$$\underline{p = a \vee b}$$

$$\begin{aligned} p_a &= p_{a=true} \oplus p_{a=false} \\ &= (true \vee b) \text{ XOR } (false \vee b) \\ &= true \text{ XOR } b \\ &= \neg b \end{aligned}$$

- Pour que la proposition majeure a détermine le prédicat p, la seule proposition mineure b doit être False.
- Par symétrie, il est clair que p_b est $\neg a$

Exemples

$$\underline{p = a \wedge b}$$

$$\begin{aligned} p_a &= p_{a=true} \oplus p_{a=false} \\ &= (true \wedge b) \oplus (false \wedge b) \\ &= b \oplus false \\ &= b \end{aligned}$$

- Nous avons besoin de $b = true$ pour déterminer p.
- Par symétrie, il est clair que p_b est a.

Exemples

$$\underline{p = a \vee (b \wedge c)}$$

$$\begin{aligned} p_a &= p_{a=true} \oplus p_{a=false} \\ &= (true \vee (b \wedge c)) \oplus (false \vee (b \wedge c)) \\ &= true \oplus (b \wedge c) \\ &= \neg (b \wedge c) \\ &= \neg b \vee \neg c \end{aligned}$$

"Not b ∨ Not c" signifie que b ou c peuvent être false

Un exemple plus subtil

- *a détermine toujours la valeur de ce prédicat*
- *b ne détermine jamais la valeur – b n'est pas pertinent !*

$$\underline{p = (a \wedge b) \vee (a \wedge \neg b)}$$

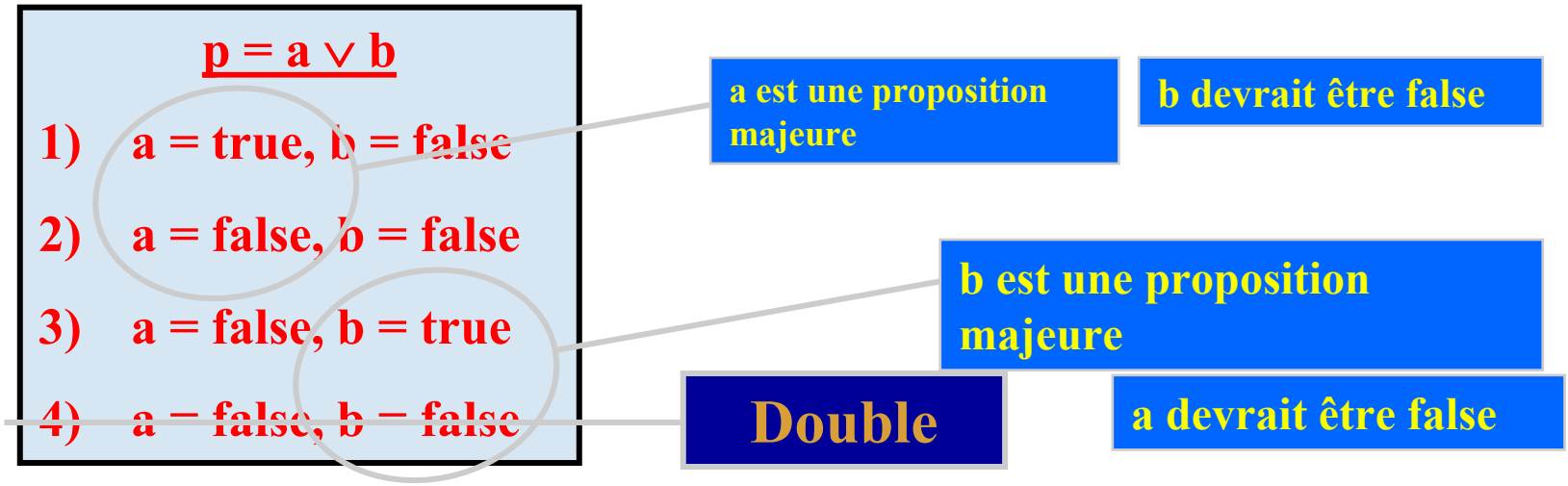
$$\begin{aligned} p_a &= p_{a=true} \oplus p_{a=false} \\ &= ((true \wedge b) \vee (true \wedge \neg b)) \oplus ((false \wedge b) \vee (false \wedge \neg b)) \\ &= (b \vee \neg b) \oplus false \\ &= true \oplus false \\ &= true \end{aligned}$$

$$\underline{p = (a \wedge b) \vee (a \wedge \neg b)}$$

$$\begin{aligned} p_b &= p_{b=true} \oplus p_{b=false} \\ &= ((a \wedge true) \vee (a \wedge \neg true)) \oplus ((a \wedge false) \vee (a \wedge \neg false)) \\ &= (a \vee false) \oplus (false \vee a) \\ &= a \oplus a \\ &= false \end{aligned}$$

Couverture de la clause active

Couverture de clause active (ACC) : Pour chaque p dans P et chaque clause majeure ci dans Cp, choisissez les clauses mineures cj, j != i, de sorte que ci détermine p. TR a deux exigences pour chaque ci : ci est évalué à vrai et ci est évalué à faux.



- Ambiguïté : les clauses mineures doivent-elles avoir les mêmes valeurs lorsque la clause majeure est vraie et fausse ?

Résoudre l'ambiguïté

- Cette question a semé la confusion parmi les testeurs pendant des années
- Un examen attentif de ces éléments conduit à trois critères distincts :
  - Les clauses mineures n'ont pas besoin d'être identiques
  - Les clauses mineures doivent être identiques
  - Les clauses mineures forcent le prédicat à devenir à la fois vrai et faux

$p = a \vee (b \wedge c)$

Proposition majeure : a

a = true, b = false, c = true

a = false, b = false, **c = false**

Est-ce autorisé ?

Pour que a soit une clause majeure, b ou c doit être false.

Couverture générale de la clause active

Couverture générale des clauses actives (General Active Clause Coverage (GACC)) : Pour chaque p dans P et chaque clause majeure ci dans Cp, choisissez les clauses mineures cj, j != i, de sorte que ci détermine p. TR a deux exigences pour chaque ci : ci est évalué à true et ci est évalué à false. Les valeurs choisies pour les clauses mineures cj n'ont pas besoin d'être les mêmes lorsque ci est true et lorsque ci est false.

- C'est compliqué !
- Il est possible de satisfaire GACC sans satisfaire la couverture des prédicats
- Nous voulons vraiment que les prédicats soient à la fois true et false !

Couverture générale de la clause active

Exemple: P= a↔b

a	b	a↔b
T	T	T
T	F	F
F	T	F
F	F	T

- La clause a détermine P pour toute attribution de valeurs de vérité à b. Ainsi, lorsque a est true, nous choisissons que b est également true et lorsque a est false, nous choisissons que b est également false.
- Nous nous retrouvons avec deux entrées de test : {TT, FF}
- P est évalué comme true dans ces deux cas.
- **La couverture du prédicat n'est pas atteinte**

Correlated Active Clause Coverage

Correlated Active Clause Coverage (CACC) : pour chaque p dans P et chaque clause majeure ci dans Cp, choisissez les clauses mineures cj, j != i, de sorte que ci détermine p. TR a deux exigences pour chaque ci : ci est évalué à true et ci est évalué à false. Les valeurs choisies pour les clauses mineures cj doivent faire en sorte que p soit true pour une valeur de la clause majeure ci et false pour l'autre.

- Permet implicitement aux clauses mineures d'avoir des valeurs différentes
- Satisfait explicitement (**subsume**) la couverture du prédicat

Correlated Active Clause Coverage

Exemple: P= a↔b

a	b	a↔b
T	T	T
T	F	F
F	T	F
F	F	T

- Le CACC peut être satisfait par rapport à la clause a avec l'ensemble de test {TT, FT} et par rapport à la clause b avec l'ensemble de test {TT, TF}.
- Ensemble de test CACC : {TT, FT, TF}

Restricted Active Clause Coverage

Restricted Active Clause Coverage (RACC) : pour chaque p dans P et chaque clause majeure ci dans Cp, choisissez les clauses mineures cj, j != i, de sorte que ci détermine p. TR a deux exigences pour chaque ci : ci est évalué à true et ci est évalué à false. Les valeurs choisies pour les clauses mineures cj doivent être les mêmes lorsque ci est true et lorsque ci est false.

- Il n'y a aucune raison logique à une telle restriction

CACC et RACC

	a	b	c	a ∧ (b ∨ c)
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F

major clause

CACC peut être satisfait en choisissant l'une des rangées 1, 2, 3 ET l'une des rangées 5, 6, 7 - un total de neuf paires

	a	b	c	a ∧ (b ∨ c)
1	T	T	T	T
5	F	T	T	F
2	T	T	F	T
6	F	T	F	F
3	T	F	T	T
7	F	F	T	F

major clause

RACC ne peut être satisfait que par l'une des trois paires ci-dessus

Exigences de test irréalisables

- Considérez le prédicat :

  $(a > b \wedge b > c) \vee c > a$

- $(a > b) = true$, $(b > c) = true$, $(c > a) = true$ est <u>irréalisable</u>

- Comme pour les critères basés sur des graphiques, les exigences de test irréalisables doivent être reconnues et <u>ignorées</u>
- Reconnaître les exigences de test irréalisables est difficile et, en général, <u>indécidable</u>
- Les tests de logiciels sont inexacts : c'est de l'ingénierie, pas de la science